

**ANALISIS UNIB BELAKANG NAVIGATION ALGORITMA DIJKSTRA
TERBOBOT DINAMIS (WAKTU & HARI)**



Disusun Oleh :

Nama : Angelita Rahmatun Annisa
NPM : G1A024067

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas selesainya laporan project mata kuliah Kecerdasan Buatan ini. Laporan Analisis berjudul Analisis Unib Belakang Navigation Algoritma Dijkstra Terbobot Dinamis (Waktu & Hari) ini disusun untuk menjelaskan implementasi algoritma Dijkstra dengan pembobotan dinamis berbasis kondisi temporal serta analisis efisiensi jalurnya.

Terima kasih penulis ucapkan kepada Ibu Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP. selaku dosen pengampu atas segala ilmu serta bimbingannya. Penulis menyadari laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan. Semoga laporan ini bermanfaat bagi pengembangan ilmu keamanan informasi.

Bengkulu, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	1
DAFTAR ISI	2
DAFTAR TABEL	3
BAB I	4
PENDAHULUAN	4
1.1 Deskripsi Proyek	4
1.2 Tujuan Analisis	4
1.3 Ruang Lingkup Analisis	5
1.4 Metode Analisis	5
BAB II	6
ANALISIS SISTEM	6
2.1 Analisis Representasi Graf	6
2.2 Analisis Struktur Simpul (Node)	6
2.3 Analisis Struktur Sisi (Edge) dan Bobot	9
2.4 Analisis Graf Dinamis Berdasarkan Waktu dan Hari	9
2.4.1 Pengaruh Hari terhadap Graf	9
2.4.2 Pengaruh Waktu terhadap Graf	10
2.5 Analisis Algoritma Dijkstra	10
2.6 Analisis Alur Proses Sistem	10
2.7 Analisis Visualisasi Peta	11
2.8 Analisis Keunggulan dan Keterbatasan Sistem	11
BAB III	12
ANALISIS IMPLEMENTASI PROGRAM	12
3.1 Struktur Komponen dan Antarmuka Web	12
3.2 Deklarasi Objek Simpul (Nodes).....	13
3.3 Fungsi Graf Dinamis.....	16
3.4 Implementasi Algoritma Dijkstra di Sisi User	17
3.5 Pemrosesan Alur Navigasi Utama dan Visualisasi Peta	18
BAB IV	20
PENUTUP	20
4.1 Kesimpulan	20
4.2 Saran	20

DAFTAR TABEL

Tabel 1 Daftar Simpul (Node) Pada Sistem Unib Belakang Navigation	9
---	---

BAB I

PENDAHULUAN

1.1 Deskripsi Proyek

Universitas Bengkulu merupakan salah satu perguruan tinggi yang memiliki area kampus yang luas dan terdiri atas berbagai fasilitas akademik maupun nonakademik. Secara umum, lingkungan kampus sering dibagi menjadi dua kawasan utama, yaitu UNIB Depan dan UNIB Belakang. Pada area UNIB Belakang terdapat berbagai gedung perkuliahan, laboratorium, fasilitas olahraga, gedung administrasi, dan berbagai jalur penghubung yang saling terhubung melalui jaringan jalan kampus.

Luasnya area kampus serta banyaknya titik lokasi menyebabkan pengguna sering mengalami kesulitan dalam menentukan jalur tercepat menuju lokasi tujuan tertentu. Permasalahan tersebut semakin kompleks karena terdapat beberapa akses yang memiliki aturan operasional berbeda berdasarkan waktu dan hari tertentu. Sebagai contoh, beberapa gerbang hanya dibuka pada jam operasional tertentu dan beberapa ruas jalan menerapkan sistem satu arah pada waktu tertentu. Kondisi tersebut menyebabkan jalur yang tersedia dapat berubah sesuai dengan kondisi operasional kampus.

Untuk mengatasi permasalahan tersebut, dikembangkan sebuah sistem bernama UNIB Belakang Navigation Algoritma Dijkstra Terbobot Dinamis (Waktu & Hari). Sistem ini merupakan aplikasi navigasi berbasis web yang dirancang untuk membantu pengguna menemukan jalur terpendek menuju lokasi tujuan di area UNIB Belakang. Sistem memanfaatkan Algoritma Dijkstra sebagai metode pencarian jalur terpendek dan menerapkan konsep graf dinamis untuk menyesuaikan kondisi jalur berdasarkan waktu dan hari operasional.

Lokasi-lokasi yang terdapat di lingkungan kampus direpresentasikan ke dalam bentuk graf berbobot yang terdiri atas simpul (*node*) dan sisi (*edge*). Setiap simpul merepresentasikan lokasi tertentu, seperti gedung, gerbang, atau persimpangan jalan, sedangkan sisi merepresentasikan hubungan antar lokasi yang dapat dilalui pengguna. Bobot yang digunakan menunjukkan estimasi jarak antar lokasi sehingga sistem dapat menghitung lintasan dengan total jarak minimum.

Selain proses pencarian jalur, sistem juga dilengkapi dengan visualisasi peta interaktif menggunakan Leaflet dan OpenStreetMap. Melalui visualisasi tersebut, pengguna dapat melihat lokasi yang dipilih, jalur yang direkomendasikan, serta urutan perjalanan yang harus dilalui menuju tujuan. Dengan demikian, sistem tidak hanya memberikan informasi berupa teks, tetapi juga menyajikan informasi secara visual sehingga lebih mudah dipahami oleh pengguna.

1.2 Tujuan Analisis

Analisis dilakukan terhadap struktur data, representasi graf, penerapan algoritma, mekanisme graf dinamis, serta implementasi sistem secara keseluruhan.

Tujuan dari analisis ini adalah sebagai berikut:

1. Menganalisis penerapan Algoritma Dijkstra dalam menentukan jalur terpendek pada sistem navigasi UNIB Belakang.
2. Menganalisis implementasi graf dinamis yang digunakan untuk menyesuaikan kondisi jalur berdasarkan waktu dan hari operasional kampus.
3. Menganalisis struktur sistem, representasi data, serta proses visualisasi jalur pada peta interaktif yang digunakan dalam aplikasi.

1.3 Ruang Lingkup Analisis

Agar pembahasan lebih terarah dan tidak melebar, analisis pada dokumen ini dibatasi pada beberapa aspek berikut:

1. Analisis dilakukan pada sistem navigasi area UNIB Belakang yang telah dikembangkan menggunakan teknologi web berbasis HTML, CSS, dan JavaScript.
2. Analisis difokuskan pada implementasi Algoritma Dijkstra sebagai metode pencarian jalur terpendek.
3. Analisis mencakup representasi lingkungan kampus ke dalam bentuk graf berbobot yang terdiri atas simpul dan sisi.
4. Analisis mencakup mekanisme graf dinamis yang mengatur perubahan akses jalur berdasarkan waktu dan hari operasional.
5. Analisis mencakup proses visualisasi lokasi dan jalur menggunakan pustaka Leaflet yang terintegrasi dengan OpenStreetMap.

1.4 Metode Analisis

Metode yang digunakan dalam penyusunan dokumen ini adalah metode analisis deskriptif, yaitu dengan mengamati dan menjelaskan komponen-komponen yang terdapat pada sistem secara sistematis.

Proses analisis dilakukan dengan mempelajari struktur data yang digunakan dalam program, hubungan antar simpul pada graf, mekanisme pembentukan graf dinamis, proses pencarian jalur menggunakan Algoritma Dijkstra, serta visualisasi hasil pencarian pada peta interaktif.

Hasil analisis kemudian digunakan untuk menjelaskan bagaimana sistem bekerja dalam menghasilkan rute terpendek yang sesuai dengan kondisi operasional kampus. Selain itu, analisis juga digunakan untuk mengevaluasi kelebihan dan keterbatasan sistem yang telah dikembangkan sehingga dapat menjadi dasar bagi pengembangan lebih lanjut di masa mendatang.

BAB II

ANALISIS SISTEM

2.1 Analisis Representasi Graf

Salah satu komponen utama dalam pengembangan sistem UNIB Belakang Navigation adalah representasi lingkungan kampus ke dalam bentuk graf (*graph*). Graf digunakan karena mampu menggambarkan hubungan antar lokasi secara terstruktur dan dapat diproses menggunakan algoritma pencarian jalur terpendek.

Dalam sistem ini, setiap lokasi penting yang terdapat di area UNIB Belakang direpresentasikan sebagai simpul (*node*). Simpul tersebut meliputi gedung, gerbang, persimpangan jalan, maupun titik-titik penting lainnya yang dapat menjadi lokasi awal, lokasi tujuan, atau titik perantara dalam proses navigasi.

Contoh simpul yang digunakan dalam sistem antara lain Gerbang Masuk, Gerbang Keluar, Gedung Rektorat UNIB, Gedung GB5, Gedung GLT, Gedung Dekanat Teknik, Danau, GSG, serta berbagai simpang yang menghubungkan jalur-jalur dalam lingkungan kampus.

Setiap simpul memiliki informasi berupa nama lokasi dan koordinat geografis dalam format lintang (*latitude*) dan bujur (*longitude*). Koordinat tersebut digunakan untuk menampilkan lokasi pada peta interaktif serta membentuk jalur visual hasil pencarian rute.

Selain simpul, sistem juga memiliki sisi (*edge*) yang merepresentasikan hubungan antar lokasi. Sisi menunjukkan bahwa dua simpul dapat dihubungkan melalui suatu jalur yang dapat dilalui pengguna. Setiap sisi diberikan bobot yang menunjukkan estimasi jarak antar lokasi dalam satuan meter.

Sebagai contoh, hubungan antara Gerbang Masuk dan Simpang Rektorat Gerbang Masuk diberikan bobot sebesar 20 meter. Nilai bobot tersebut digunakan sebagai dasar perhitungan oleh algoritma Dijkstra dalam menentukan jalur terpendek.

Melalui representasi graf berbobot ini, lingkungan kampus dapat dimodelkan ke dalam bentuk yang lebih sederhana namun tetap mampu menggambarkan kondisi nyata yang ada di lapangan.

2.2 Analisis Struktur Simpul (Node)

Pada implementasi sistem, seluruh lokasi disimpan dalam sebuah struktur data yang berisi nama lokasi dan koordinat geografis masing-masing.

Struktur simpul digunakan untuk menyimpan seluruh titik yang dapat digunakan sebagai lokasi awal maupun tujuan navigasi. Setiap simpul memiliki identitas unik berupa nama lokasi sehingga memudahkan pengguna dalam memilih tujuan yang diinginkan.

No	Nama Simpul	Kategori
1	Gerbang Keluar	Gerbang
2	Gerbang Masuk	Gerbang
3	Gerbang Rektorat	Gerbang
4	Gedung Rektorat UNIB	Gedung
5	Gedung GB5	Gedung
6	Gedung Dekanat Teknik	Gedung
7	Laboratorium Fakultas Teknik	Gedung
8	Gedung Sekre Teknik	Gedung
9	Laboratorium Terpadu Teknik Informatika	Gedung
10	Gedung GLT	Gedung
11	Danau	Landmark
12	GSG	Gedung
13	Simpang PKM Belakang	Simpang
14	Jalan Belakang PKM	Jalan
15	Jalan Dekanat FMIPA	Jalan
16	Jalan Jurusan Budidaya Pertanian	Jalan
17	Jalan Belakang GLT	Jalan
18	Jalan Teknologi Pertanian	Jalan
19	Jalan Dekat Mushola Danau	Jalan
20	Jalan depan FISIP	Jalan
21	Simpang Masuk dan Keluar Laboratorium Teknik Bagian Dalam	Simpang
22	Jalan ke GB5	Jalan
23	Simpang Rektorat Gerbang Masuk	Simpang
24	Simpang UPA TIK	Simpang
25	Simpang Perpustakaan Danau	Simpang
26	Simpang Ujung Gerbang Masuk	Simpang
27	Simpang Dekanat Teknik dan Gerbang Keluar	Simpang

28	Simpang LAB FKIP	Simpang
29	Simpang Sekre Teknik dan Laboratorium	Simpang
30	Jalan Keluar	Jalan
31	Simpang GB5 dan Gedung FKIK	Simpang
32	Jalan Samping Kuburan	Jalan
33	Simpang Masuk dan Keluar Laboratorium Teknik Bagian Luar	Simpang
34	Jalan Keluar 2	Jalan
35	Jalan Depan GSG	Jalan
36	Simpang GLT dan Teknologi Pertanian	Simpang
37	Jalan Danau 1	Jalan
38	Jalan Danau 2	Jalan
39	Jalan Danau 4	Jalan
40	Jalan Danau 5	Jalan
41	Jalan Danau 6	Jalan
42	Jalan Danau 7	Jalan
43	Jalan GLT 1	Jalan
44	Jalan GLT 2	Jalan
45	Jalan GLT 3	Jalan
46	Jalan GLT 4	Jalan
47	Jalan GLT 5	Jalan
48	Jalan GLT 6	Jalan
49	Jalan Samping Rektorat	Jalan
50	Jalan Simpang Rektorat Belakang	Jalan
51	Jalan Belakang Parkiran Rektorat	Jalan
52	Simpang GLT REKTORAT	Simpang
53	Simpang Belokan Kiri Gerbang Rektorat	Simpang
54	Simpang Belakang Rektorat dan Parkiran Belakang	Simpang

55	Simpang Rektorat dan GLT	Simpang
----	--------------------------	---------

Tabel 1 Daftar Simpul (Node) Pada Sistem Unib Belakang Navigation

Berdasarkan Tabel 1, simpul dalam sistem tidak hanya terdiri atas lokasi tujuan seperti gedung dan fasilitas kampus, tetapi juga mencakup simpang dan ruas jalan yang berfungsi sebagai titik perantara dalam proses pencarian jalur. Pendekatan ini memungkinkan Algoritma Dijkstra melakukan perhitungan lintasan secara lebih akurat karena setiap perubahan arah dan percabangan jalan direpresentasikan sebagai simpul tersendiri dalam graf. Dengan demikian, model graf yang dibangun dapat menggambarkan kondisi nyata lingkungan UNIB Belakang dengan tingkat detail yang lebih tinggi dibandingkan jika hanya menggunakan gedung sebagai simpul utama.

2.3 Analisis Struktur Sisi (Edge) dan Bobot

Sisi (*edge*) digunakan untuk menghubungkan dua simpul yang memiliki akses langsung. Setiap sisi diberikan bobot berupa estimasi jarak tempuh yang digunakan sebagai dasar pencarian jalur terpendek.

Bobot yang digunakan dalam sistem bersifat non-negatif sehingga sesuai dengan karakteristik Algoritma Dijkstra. Semakin kecil bobot suatu jalur, maka semakin besar kemungkinan jalur tersebut dipilih sebagai bagian dari rute terpendek.

Sebagai contoh, hubungan antara Simpang UPA TIK dan Simpang LAB FKIP memiliki bobot sebesar 30 meter, sedangkan hubungan antara Jalan Belakang GLT dan Jalan Jurusan Budidaya Pertanian memiliki bobot sebesar 95 meter.

Dalam proses pencarian rute, sistem akan menjumlahkan seluruh bobot pada jalur yang dilalui. Jalur dengan total bobot paling kecil akan dipilih sebagai solusi akhir yang direkomendasikan kepada pengguna.

Penggunaan bobot jarak memberikan keuntungan karena sistem mampu menghasilkan rute yang lebih efisien dibandingkan hanya memilih jalur berdasarkan jumlah simpul yang dilewati.

2.4 Analisis Graf Dinamis Berdasarkan Waktu dan Hari

Graf dinamis merupakan graf yang dapat berubah sesuai kondisi tertentu. Pada sistem ini, perubahan graf dilakukan berdasarkan dua parameter utama, yaitu hari dan waktu.

Penerapan graf dinamis pada sistem ini diperlukan karena beberapa akses jalan di lingkungan kampus tidak selalu tersedia setiap saat. Terdapat gerbang yang hanya dibuka pada hari kerja dan beberapa jalur yang menerapkan sistem satu arah pada waktu tertentu.

2.4.1 Pengaruh Hari terhadap Graf

Pada hari kerja (Senin–Jumat), Gerbang Rektorat dapat digunakan sebagai jalur alternatif menuju area rektorat dan sekitarnya.

Namun pada akhir pekan (Sabtu–Minggu), akses tersebut dianggap tidak tersedia sehingga sistem menghapus hubungan yang melewati gerbang tersebut dari graf yang digunakan dalam proses pencarian.

Kondisi ini menyebabkan hasil pencarian rute pada hari kerja dapat berbeda dengan hasil pencarian pada akhir pekan meskipun lokasi awal dan tujuan yang dipilih sama.

2.4.2 Pengaruh Waktu terhadap Graf

Sebelum pukul 17.00, beberapa jalur keluar kampus menerapkan aturan satu arah sehingga kendaraan hanya dapat bergerak pada arah tertentu. Untuk merepresentasikan kondisi tersebut, sistem memberikan nilai bobot tak hingga (*Infinity*) pada arah yang tidak dapat dilalui.

Setelah kondisi berubah, hubungan antar simpul akan disesuaikan kembali sehingga sistem dapat menghasilkan rute yang sesuai dengan aturan akses yang berlaku.

Dengan mekanisme ini, sistem mampu menghasilkan jalur yang lebih realistis dibandingkan sistem navigasi yang hanya menggunakan graf statis.

2.5 Analisis Algoritma Dijkstra

Algoritma Dijkstra digunakan sebagai metode utama dalam menentukan jalur terpendek pada sistem navigasi. Algoritma ini bekerja dengan mencari lintasan yang memiliki total bobot minimum dari suatu titik awal menuju titik tujuan. Proses pencarian dilakukan secara bertahap hingga ditemukan jalur terbaik.

Tahapan kerja algoritma dapat dijelaskan sebagai berikut:

1. Menentukan simpul awal yang dipilih pengguna.
2. Memberikan nilai jarak awal sebesar nol pada simpul awal.
3. Memberikan nilai tak hingga pada seluruh simpul lainnya.
4. Memilih simpul dengan nilai jarak terkecil yang belum diproses.
5. Memperbarui jarak seluruh simpul tetangga yang terhubung.
6. Mengulangi proses hingga simpul tujuan ditemukan.
7. Menyusun kembali lintasan berdasarkan jalur yang memiliki total bobot minimum.

Algoritma Dijkstra dipilih karena mampu menghasilkan solusi optimal pada graf berbobot non-negatif serta memiliki tingkat akurasi yang tinggi dalam pencarian jalur terpendek.

2.6 Analisis Alur Proses Sistem

Proses kerja sistem dimulai ketika pengguna memilih lokasi awal, lokasi tujuan, hari, dan waktu melalui antarmuka aplikasi. Data tersebut kemudian digunakan untuk membentuk graf dinamis sesuai kondisi operasional yang berlaku pada saat pencarian dilakukan.

Setelah graf terbentuk, sistem menjalankan Algoritma Dijkstra untuk menghitung jalur dengan total bobot minimum dari titik awal menuju titik tujuan. Hasil perhitungan berupa urutan lokasi yang harus dilalui serta total jarak perjalanan. Informasi tersebut kemudian diteruskan ke modul visualisasi untuk ditampilkan pada peta interaktif.

2.7 Analisis Visualisasi Peta

Visualisasi peta merupakan bagian penting dalam sistem karena berfungsi untuk menyampaikan hasil perhitungan jalur secara lebih mudah dipahami. Sistem menggunakan Leaflet sebagai pustaka utama untuk menampilkan peta interaktif. Leaflet dipilih karena memiliki performa yang ringan, mudah digunakan, dan mendukung integrasi dengan OpenStreetMap.

Seluruh simpul yang terdapat dalam graf ditampilkan pada peta menggunakan penanda (*marker*). Setiap marker mewakili lokasi tertentu yang dapat dipilih sebagai titik awal maupun tujuan. Setelah proses pencarian jalur selesai, sistem akan menggambar lintasan hasil perhitungan menggunakan garis (*polyline*) yang menghubungkan seluruh simpul yang dilalui.

Selain visualisasi jalur, sistem juga menampilkan informasi berupa total jarak perjalanan dan daftar langkah perjalanan yang harus dilalui pengguna. Dengan adanya visualisasi tersebut, pengguna dapat memahami hasil pencarian secara lebih intuitif dibandingkan hanya melihat data dalam bentuk teks.

2.8 Analisis Keunggulan dan Keterbatasan Sistem

Berdasarkan hasil implementasi dan pengujian yang dilakukan, sistem memiliki beberapa keunggulan.

Pertama, sistem mampu menentukan jalur terpendek secara otomatis menggunakan Algoritma Dijkstra. Kedua, sistem menerapkan graf dinamis sehingga hasil pencarian dapat menyesuaikan kondisi waktu dan hari operasional kampus. Ketiga, sistem menyediakan visualisasi peta interaktif yang memudahkan pengguna dalam memahami rute yang direkomendasikan. Keempat, sistem dapat diakses melalui browser tanpa memerlukan instalasi aplikasi tambahan.

Meskipun demikian, sistem juga memiliki beberapa keterbatasan. Bobot jalur masih menggunakan estimasi jarak statis dan belum mempertimbangkan kondisi lalu lintas secara real-time. Selain itu, sistem hanya mencakup area UNIB Belakang dan belum mengintegrasikan seluruh wilayah Universitas Bengkulu. Sistem juga belum memanfaatkan GPS untuk mendeteksi lokasi pengguna secara otomatis.

Keterbatasan tersebut dapat menjadi dasar pengembangan sistem pada penelitian dan implementasi selanjutnya.

BAB III

ANALISIS IMPLEMENTASI PROGRAM

3.1 Struktur Komponen dan Interface Web

Implementasi antarmuka program UNIB Belakang Navigation dirancang menggunakan dokumen HTML5 tunggal yang menerapkan *Single Page Application*. Penataan komponen visual dan tata letak halaman seluruhnya dikelola oleh *utility-first classes* dari Tailwind CSS v4, sedangkan mesin visualisasi serta pemetaan spasialnya digerakkan oleh pustaka Leaflet.js. Secara struktural, halaman web dibagi menjadi tiga komponen utama yang diatur secara hierarkis melalui kode pembungkus berikut:

```
<div class="flex-grow w-full flex flex-col md:flex-row h-[calc(100vh-88px)] p-4 gap-4 overflow-hidden">
```

```
<div id="sidePanel" class="hidden w-full md:w-80 lg:w-96 bg-white p-5 rounded-2xl shadow-md border border-slate-200 flex-col gap-4 h-full shrink-0 overflow-hidden">
```

```
<h2 class="text-xs font-bold text-slate-400 uppercase tracking-widest border-b pb-2 flex items-center gap-2">
```

 Detail Hasil Pencarian

```
</h2>
```

```
<div id="hasilInfo" class="flex-grow overflow-y-auto space-y-4 pr-1"></div>
```

```
</div>
```

```
<div id="mapWrapper" class="flex-grow bg-white p-2 rounded-2xl shadow-xl border border-slate-200 h-full relative">
```

```
<div id="map" class="rounded-xl"></div>
```

```
</div>
```

```
</div>
```

Berdasarkan struktur kode di atas, komponen pertama adalah bilah menu atas `<nav>` yang didesain menggunakan properti lekat `sticky top-0 z-50` agar tetap berada di posisi teratas browser meskipun dokumen digulirkan. Di dalam bilah menu tersebut diletakkan identitas pengembang berupa nama Angelita Rahmatun Annisa dan NPM G1A024067, selektor kondisi temporal berupa pilihan hari kerja atau libur serta jam simulasi, formulir input teks untuk menentukan simpul awal dan simpul akhir perjalanan, serta tombol eksekusi utama pencarian jalur.

Komponen kedua adalah panel informasi samping hasil pencarian `#sidePanel`. Pada fase awal inisialisasi aplikasi, panel ini secara sengaja diberikan kelas tersembunyi `hidden` agar tidak

memakan ruang visual pada layar utama. Sesaat setelah pengguna memicu fungsi pencarian rute dan algoritma berhasil menemukan lintasan optimal, manipulasi DOM via JavaScript akan mengubah properti komponen ini menjadi kelas `flex`, sehingga memunculkan ringkasan akumulasi jarak dalam satuan meter dan daftar urutan rute langkah perjalanan yang dilengkapi penomoran rapi pada elemen `#hasilInfo`. Komponen ketiga adalah kontainer peta utama `#mapWrapper` yang diatur mengisi seluruh sisa ruang visual di sisi kanan layar melalui perhitungan tinggi `viewport` otomatis `h-[calc(100vh-88px)]`. Kontainer inilah yang membungkus elemen peta tempat pustaka Leaflet melakukan perendangan petak gambar berbasis data spasial OpenStreetMap secara interaktif.

3.2 Deklarasi Objek Simpul (Nodes)

Representasi matematis dari objek fisik di dunia nyata (seperti gedung perkuliahan, dekanat, laboratorium, persimpangan jalan, dan gerbang akses) dimodelkan ke dalam sebuah objek JavaScript bernama data master simpul `const nodes`. Data koordinat absolut ini disisipkan secara manual berdasarkan pemetaan titik riil di lapangan guna memastikan akurasi navigasi spasial di lingkungan Universitas Bengkulu area belakang.

```
const nodes = {  
  "Gerbang Keluar": [-3.759356082679988, 102.27620478124236],  
  "Gerbang Masuk": [-3.759577166889533, 102.27511893639122],  
  "Gerbang Rektorat": [-3.7606332905402904, 102.2726371975201],  
  "Gedung Rektorat UNIB": [-3.7593690314480206, 102.27235244963148],  
  "Gedung GB5": [-3.7554612742834395, 102.27638350369625],  
  "Gedung Dekanat Teknik": [-3.758711950725617, 102.27671505155469],  
  "Laboratorium Fakultas Teknik": [-3.7589287422927424, 102.27676333131582],  
  "Gedung Sekre Teknik": [-3.758246250133572, 102.27736414613804],  
  "Laboratorium Terpadu Teknik Informatika": [-3.7585808051783998,  
102.27729172649424],  
  "Gedung GLT": [-3.758042440780505, 102.27185754507934],  
  "Danau": [-3.758389863319991, 102.27278942221876],  
  "GSG": [-3.7577015441649313, 102.27656173115679],  
  "Simpang PKM Belakang": [-3.756043831170103, 102.27561720401069],  
  "Jalan Belakang PKM": [-3.756074020901313, 102.27582558989614],  
  "Jalan Dekanat FMIPA": [-3.755608318900689, 102.27472588419562],  
  "Jalan Jurusan Budidaya Pertanian": [-3.75627743089022, 102.27295026183013],  
  "Jalan Belakang GLT": [-3.7567539129628336, 102.27150790546798],  
  "Jalan Teknologi Pertanian": [-3.75771066673857, 102.2712407307806],  
}
```

"Jalan Dekat Mushola Danau": [-3.7578554506276025, 102.27378528177347],

"Jalan depan FISIP": [-3.759976263597784, 102.27420444751262],

"Simpang Masuk dan Keluar Laboratorium Teknik Bagian Dalam": [-3.7592177976379157, 102.27697790803725],

"Jalan ke GB5": [-3.757361391237785, 102.27718010544687],

"Simpang Rektorat Gerbang Masuk": [-3.759424394475821, 102.27504039577254],

"Simpang UPA TIK": [-3.7588116661806517, 102.27480011556756],

"Simpang Perpustakaan Danau": [-3.757622536129889, 102.27414480591212],

"Simpang Ujung Gerbang Masuk": [-3.755859423845089, 102.27370209863312],

"Simpang Dekanat Teknik dan Gerbang Keluar": [-3.7584095130154953, 102.2762751692775],

"Simpang LAB FKIP": [-3.7585115124344806, 102.27578350097083],

"Simpang Sekre Teknik dan Laboratorium": [-3.7581900448647274, 102.27717102708827],

"Jalan Keluar": [-3.7588307575600095, 102.2762976586807],

"Simpang GB5 dan Gedung FKIK": [-3.7557615850107857, 102.27683979338227],

"Jalan Samping Kuburan": [-3.7583939791096013, 102.27446766507919],

"Simpang Masuk dan Keluar Laboratorium Teknik Bagian Luar": [-3.758758959397839, 102.27654564408476],

"Jalan Keluar 2": [-3.7589622141455763, 102.27629233362865],

"Jalan Devan GSG": [-3.7581831402373593, 102.27665537652081],

"Simpang GLT dan Teknologi Pertanian": [-3.7584304360859755, 102.27106079578179],

"Jalan Danau 1": [-3.7577741804787284, 102.2739305205689],

"Jalan Danau 2": [-3.7578972968167674, 102.27372667268271],

"Jalan Danau 5": [-3.758021509038929, 102.27349696969101],

"Jalan Danau 6": [-3.75806320236126, 102.27339512321822],

"Jalan Danau 7": [-3.758138771502886, 102.27324627068108],

"Jalan Danau 4": [-3.7582514316396662, 102.27296680509195],

"Jalan GLT 1": [-3.7584149896911225, 102.27247850496857],

"Jalan GLT 3": [-3.7584202013577688, 102.27247850496691],

"Simpang GLT": [-3.758475525166938, 102.27210419746689],

"Jalan GLT 2": [-3.758519222948783, 102.27161150422778],

"Jalan GLT 4": [-3.7585270404428113, 102.27147421205628],

"Jalan GLT 5": [-3.7585368419887195, 102.27141082658717],

```

    "Jalan GLT 6": [-3.7585597941193756, 102.27119997855868],
    "Jalan Samping Rektorat": [-3.758844951793753, 102.27133207929076],
    "Jalan Simpang Rektorat Belakang": [-3.759045600703256, 102.27155927526779],
    "Jalan Belakang Parkiran Rektorat": [-3.7589700316401893, 102.2719927756391],
    "Simpang GLT REKTORAT": [-3.7585926961478626, 102.27102781201282],
    "Simpang Belokan Kiri Gerbang Rektorat": [-3.760652499089209,
102.27228400534246],
    "Simpang Belakang Rektorat dan Parkiran Belakang": [-3.7590377112562137,
102.2715407994551],
    "Simpang Rektorat dan GLT": [-3.7585921489104916, 102.2710313438033]
  };

```

Pada saat aplikasi pertama kali dimuat di browser, data mentah di dalam objek master simpul tersebut langsung diproses secara kolektif menggunakan metode perulangan JavaScript berikut:

```

Object.keys(nodes).forEach(name => {
  let isGedung = !name.startsWith("Simpang") && !name.startsWith("Jalan") &&
!name.startsWith("Jembatan");
  let markerColor = isGedung ? "#11206a" : "#94a3b8";
  let markerRadius = isGedung ? 6 : 4;
  if (name.includes("Gerbang")) { markerColor = "#a99702"; markerRadius = 7; }
  L.circleMarker(nodes[name], { radius: markerRadius, fillColor: markerColor, color:
"#f8fafc", weight: 1.5, fillOpacity: 0.85 }).addTo(map).bindPopup('<b>${name}</b>');
});

```

Program akan membaca seluruh nama simpul yang tersedia untuk digambar ke atas *layer* peta Leaflet dalam bentuk objek lingkaran penanda (**L.circleMarker**) secara dinamis. Di dalam proses perulangan tersebut, diterapkan sistem penyaringan berbasis manipulasi string untuk memilah jenis lokasi secara otomatis tanpa perlu menuliskan konfigurasi manual untuk setiap titik. Melalui logika pengondisian, apabila string nama lokasi tidak diawali dengan kata benda seperti "Simpang", "Jalan", atau "Jembatan", maka program langsung mengklasifikasikan titik tersebut sebagai bangunan gedung kuliah.

Lokasi gedung ini kemudian diberikan warna penanda biru tua institusional khas Universitas Bengkulu (**#11206a**) dengan ukuran radius standar sebesar 6 piksel. Sebaliknya, apabila nama lokasi mendeteksi adanya string kata "Gerbang", sistem secara otomatis akan memberikan warna penanda kuning emas (**#a99702**) dengan ukuran radius lingkaran terbesar, yaitu 7 piksel. Hal ini dilakukan untuk menciptakan hierarki visual yang jelas, sehingga

pengguna dapat dengan mudah membedakan antara titik transit jalan, area gedung tujuan, dan gerbang utama akses keluar masuk kampus.

3.3 Fungsi Graf Dinamis

Inti dari implementasi kecerdasan buatan yang mengatur fleksibilitas aturan lalu lintas kampus dikendalikan sepenuhnya di dalam fungsi pembuatan graf dinamis. Fungsi ini menerima kiriman parameter berupa string **hari** dan integer **jam** yang diinput oleh pengguna dari formulir web, kemudian mengembalikan sebuah struktur data daftar ketetanggaan (*adjacency list*) yang memetakan hubungan antarjalur beserta bobot jarak fisiknya dalam satuan meter.

```
function buatGrafDinamis(hari, jam) {  
    const isOneWay = jam < 17;  
  
    return {  
        "Simpang Rektorat Gerbang Masuk": {  
            "Gerbang Masuk": (isOneWay ? Infinity : 20),  
            "Simpang UPA TIK": 75,  
            "Gerbang Rektorat": (hari === "kerja" && jam < 17 ? 60 : Infinity)  
        },  
        "Simpang UPA TIK": {  
            "Simpang Rektorat Gerbang Masuk": (isOneWay ? Infinity : 75),  
            "Simpang Perpustakaan": 150,  
            "Simpang LAB FKIP": 30,  
        },  
        "Simpang Dekanat Teknik dan Gerbang Keluar": {  
            "Simpang LAB FKIP": (isOneWay ? Infinity : 55),  
            "Jalan Keluar": (jam < 17 ? 15 : Infinity)  
        },  
        // ... (daftar edges ketetanggaan lainnya)  
    };  
}
```

Di dalam fungsi ini, dideklarasikan sebuah variabel kesimpulan bertipe boolean bernama **isOneWay = jam < 17** untuk mendeteksi kondisi jam sibuk perkuliahan kampus. Apabila jam simulasi yang dimasukkan berada di bawah pukul 17:00 sore, maka kondisi satu arah dinilai aktif, sehingga program secara otomatis mengubah bobot jarak pada sisi balik arus poros jalan utama teknik menjadi nilai matematika abstrak berupa **Infinity** atau jarak tak terhingga.

Penggunaan nilai tak terhingga ini secara efektif memutus keterhubungan antar-simpul pada arah tersebut (seperti terlihat pada kunci objek "**Simpang Rektorat Gerbang Masuk**" balik menuju "**Gerbang Masuk**"), sehingga mesin pencari jalur akan mendeteksi jalur keluar tersebut sebagai jalan buntu. Logika pengondisional berbasis operator *ternary* ini juga diterapkan untuk merekayasa jadwal buka-tutup akses gerbang fisik kampus lainnya. Gerbang Rektorat dikonfigurasi hanya memuat nilai angka jarak riil (60 meter) apabila parameter hari bernilai hari kerja ("**kerja**") dan jam berada di bawah pukul 17:00 sore. Di luar parameter waktu tersebut, bobot jalurnya langsung bermutasi menjadi **Infinity** yang menyimulasikan gerbang terkunci rapat. Aturan dinamis serupa juga diterapkan pada objek "**Jalan Keluar**" dan "**Jalan Keluar 2**", di mana jalur tersebut hanya membuka arus satu arah keluar sebelum pukul 17:00 sore, dan secara otomatis akan berubah status menjadi tertutup jika waktu simulasi menunjukkan jam malam hari.

3.4 Implementasi Algoritma Dijkstra Sisi User

Proses penentuan keputusan lintasan terpendek dieksekusi secara mandiri pada memori lokal browser melalui fungsi inti algoritma Dijkstra yang ditulis sebagai berikut:

```
function dijkstra(graf, start, end) {  
  let distances = {}; let prev = {}; let pq = [];  
  Object.keys(nodes).forEach(node => { distances[node] = Infinity; prev[node] = null; });  
  distances[start] = 0;  
  pq.push({ node: start, dist: 0 });  
  
  while (pq.length > 0) {  
    pq.sort((a, b) => a.dist - b.dist);  
    let current = pq.shift(); let u = current.node;  
    if (u === end) break;  
    if (graf[u]) {  
      Object.keys(graf[u]).forEach(neighbor => {  
        let alt = distances[u] + graf[u][neighbor];  
        if (alt < distances[neighbor]) { distances[neighbor] = alt; prev[neighbor] = u;  
        pq.push({ node: neighbor, dist: alt }); }  
      });  
    }  
  }  
  let path = []; let u = end;  
  if (prev[u] || u === start) { while (u) { path.unshift(u); u = prev[u]; } }
```

```

return { path: path, distance: distances[end] };
}

```

Tahapan algoritma dimulai dengan fase inisialisasi, di mana program membuat objek penyimpanan jarak sementara **distances** yang diisi dengan nilai default awal berupa **Infinity** untuk seluruh simpul yang terdaftar, kecuali untuk simpul **start** yang diberikan nilai awal jarak nol. Simpul asal ini kemudian dimasukkan ke dalam sebuah array **pq** yang bertindak sebagai struktur antrean prioritas tiruan. Proses pencarian dijalankan menggunakan perulangan **while** yang akan terus berputar selama antrean prioritas masih memiliki elemen data di dalamnya. Pada setiap awal iterasi perulangan, array antrean akan diurutkan secara menaik menggunakan fungsi **pq.sort((a, b) => a.dist - b.dist)** berdasarkan nilai akumulasi jarak terkecil. Simpul dengan bobot paling minimum yang berada di baris terdepan antrean kemudian dikeluarkan dari array menggunakan fungsi **.shift()** untuk ditetapkan sebagai simpul aktif **u** yang siap dieksplorasi.

Setelah simpul aktif terpilih, program akan memeriksa seluruh simpul tetangga (*neighbor*) yang terhubung langsung berdasarkan cetakan graf dinamis yang aktif pada jam tersebut. Proses relaksasi sisi dilakukan dengan menjumlahkan nilai jarak simpul aktif ditambah dengan bobot jalan menuju simpul tetangganya (**distances[u] + graf[u][neighbor]**). Jika nilai penjumlahan alternatif (**alt**) tersebut terbukti menghasilkan angka yang lebih kecil dari rekam data jarak yang tersimpan pada memori sebelumnya (**distances[neighbor]**), maka rekam data jarak simpul tetangga tersebut akan diperbarui dengan nilai yang baru. Di saat yang sama, nama simpul aktif dicatat ke dalam objek penunjuk rute sebelumnya (**prev**) sebagai referensi hubungan, dan simpul tetangga tersebut dimasukkan ke dalam antrean prioritas **pq** untuk dievaluasi pada putaran berikutnya. Perulangan pencarian ini akan dihentikan secara paksa melalui perintah **break** tepat ketika simpul yang dikeluarkan dari antrean prioritas terdeteksi merupakan simpul tujuan akhir (**end**). Tahap akhir dari fungsi ini adalah proses penelusuran balik (*backtracking*), di mana program melakukan perulangan mundur dari simpul tujuan menuju simpul awal dengan membaca objek **prev**, lalu memasukkan nama lokasi ke baris terdepan array rute menggunakan fungsi **.unshift(u)** hingga menghasilkan urutan langkah perjalanan yang runut dan valid di dalam array **path**.

3.5 Pemrosesan Alur Navigasi Utama dan Visualisasi Peta

Fungsi proses pencarian jalur bertindak sebagai jembatan utama yang mengoordinasikan seluruh alur sistem, mulai dari pembacaan data formulir, pemanggilan fungsi graf dinamis, eksekusi algoritma Dijkstra, hingga penggambaran spasial lintasan pada peta Leaflet melalui runtutan operasi DOM berikut:

```

sidePanel.classList.remove('hidden');
sidePanel.classList.add('flex');

```

```

if (hasilFinal.path.length === 0 || hasilFinal.distance === Infinity) {
  infoBox.innerHTML = `<div class="text-red-700 font-bold">✖ Jalur Buntu!</div>...`;
  setTimeout(() => { map.invalidateSize(); }, 100);
  return;
}

const pointJalur = hasilFinal.path.map(namaNode => nodes[namaNode]);
routeLayer = L.polyline(pointJalur, { color: '#11206a', weight: 6, opacity: 0.9, lineCap: 'round',
lineJoin: 'round' }).addTo(map);
map.fitBounds(routeLayer.getBounds(), { padding: [50, 50] });

```

Pada saat fungsi ini dipicu, program secara instan memanipulasi properti CSS dokumen untuk memunculkan panel samping kiri ke layar dengan mengubah kelas dari **hidden** menjadi **flex**. Sebelum masuk ke tahap penggambaran, program melakukan validasi terhadap array rute yang dihasilkan. Apabila algoritma Dijkstra mengembalikan panjang array rute bernilai nol atau total jarak akumulatif bernilai tak terhingga (**Infinity**), sistem akan langsung memutus alur program dan mencetak notifikasi teks berwarna merah pada elemen **infoBox** untuk memberitahukan pengguna bahwa rute tidak dapat ditemukan akibat terhalang aturan jalan buntu atau penutupan gerbang kampus.

Jika jika rute sah berhasil ditemukan, maka kumpulan string nama lokasi yang tersimpan di dalam array langkah perjalanan akan dikonversi kembali menjadi data koordinat angka numerik dua dimensi dengan memanfaatkan metode pemetaan array **.map(namaNode => nodes[namaNode])** yang membaca objek data master simpul. Kumpulan koordinat yang telah terurut tersebut kemudian disimpan ke dalam variabel **pointJalur** lalu dilemparkan ke dalam fungsi pustaka Leaflet **L.polyline()**. Garis rute tersebut dikonfigurasi menggunakan warna biru tua kontras (**#11206a**) dengan ketebalan garis sebesar 6 piksel serta memiliki ujung sambungan yang membulat (**lineCap: 'round', lineJoin: 'round'**) demi menjaga estetika kehalusan visual alur jalan di atas peta. Tahap final dari pemrosesan ini adalah pemanggilan fungsi penyegaran ukuran peta **map.invalidateSize()** lewat waktu tunggu *timeout* 100 milidetik untuk memaksa Leaflet menghitung ulang dimensi kontainernya yang melar akibat munculnya panel samping baru. Program kemudian mengeksekusi metode **map.fitBounds()** untuk menggeser posisi kamera peta dan melakukan operasi perbesaran skala otomatis, sehingga seluruh garis lintasan rute terpendek dari titik awal hingga akhir terkunci sempurna di tengah-tengah layar monitor pengguna.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan hasil analisis dan implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem UNIB Belakang Navigation: Algoritma Dijkstra Terbobot Dinamis (Waktu & Hari) berhasil dikembangkan sebagai sistem navigasi berbasis web yang mampu membantu pengguna dalam menentukan jalur terpendek menuju lokasi tujuan di area UNIB Belakang Universitas Bengkulu.

Penerapan Algoritma Dijkstra pada sistem mampu menghasilkan rute dengan total bobot jarak minimum berdasarkan graf yang telah dibentuk dari representasi lingkungan kampus. Setiap lokasi direpresentasikan sebagai simpul (*node*) dan setiap hubungan antar lokasi direpresentasikan sebagai sisi (*edge*) yang memiliki bobot berupa estimasi jarak. Dengan pendekatan tersebut, sistem dapat melakukan pencarian jalur secara optimal dan memberikan rekomendasi rute yang lebih efisien kepada pengguna.

Selain menggunakan graf berbobot, sistem juga menerapkan konsep graf dinamis yang memungkinkan perubahan struktur graf berdasarkan kondisi waktu dan hari operasional. Penerapan mekanisme ini membuat sistem mampu menyesuaikan hasil pencarian rute terhadap kondisi nyata di lingkungan kampus, seperti aturan jalan satu arah dan akses gerbang yang hanya tersedia pada waktu tertentu. Dengan demikian, jalur yang dihasilkan tidak hanya merupakan jalur terpendek secara matematis, tetapi juga sesuai dengan kondisi operasional yang berlaku.

Penggunaan Leaflet dan OpenStreetMap sebagai media visualisasi berhasil memberikan tampilan peta interaktif yang memudahkan pengguna dalam memahami hasil pencarian jalur. Informasi rute yang ditampilkan secara visual mampu meningkatkan kemudahan penggunaan sistem dibandingkan penyajian informasi dalam bentuk teks semata.

Secara keseluruhan, sistem yang dikembangkan telah memenuhi tujuan utama proyek, yaitu menyediakan navigasi area UNIB Belakang yang mampu menentukan jalur terpendek secara dinamis berdasarkan waktu dan hari menggunakan Algoritma Dijkstra.

4.2 Saran

Meskipun sistem telah mampu menjalankan fungsi navigasi dengan baik, masih terdapat beberapa aspek yang dapat dikembangkan untuk meningkatkan kualitas dan kemampuan sistem pada masa mendatang.

Pertama, sistem dapat dikembangkan dengan menambahkan cakupan wilayah navigasi sehingga tidak hanya mencakup area UNIB Belakang, tetapi juga seluruh kawasan Universitas Bengkulu, termasuk area UNIB Depan dan fasilitas kampus lainnya.

Kedua, bobot yang digunakan pada setiap jalur saat ini masih berupa estimasi jarak statis. Pada pengembangan selanjutnya, bobot dapat disesuaikan dengan kondisi aktual di lapangan, seperti tingkat kepadatan kendaraan, kondisi jalan, maupun waktu tempuh rata-rata sehingga hasil navigasi menjadi lebih akurat.

Ketiga, sistem dapat diintegrasikan dengan teknologi Global Positioning System (GPS) untuk mendeteksi lokasi pengguna secara otomatis dan memberikan navigasi secara real-time tanpa perlu memasukkan titik awal secara manual.

Keempat, visualisasi peta dapat ditingkatkan dengan menambahkan fitur penunjuk arah, estimasi waktu tempuh, pencarian lokasi otomatis, serta informasi fasilitas kampus sehingga pengalaman pengguna menjadi lebih interaktif dan informatif.

Kelima, algoritma pencarian jalur dapat dibandingkan dengan algoritma lain seperti A* (*A-Star*) atau Floyd-Warshall untuk mengetahui metode yang paling efektif dan efisien dalam menangani permasalahan navigasi di lingkungan kampus yang memiliki kondisi jalur dinamis.

Dengan adanya pengembangan tersebut, diharapkan sistem UNIB Belakang Navigation dapat menjadi solusi navigasi kampus yang lebih lengkap, akurat, dan bermanfaat bagi seluruh civitas akademika Universitas Bengkulu.